PARTIE 1 — QUESTIONS DE COURS (Théorie + compréhension)

🧠 Q1.

Explique la différence entre :

- une **classe abstraite**
- une **interface**

➡️ Donne un exemple de chacune.

⚙️ 1️⃣ Classe abstraite

🔷 Définition :

Une **classe abstraite** est une **classe qui ne peut pas être instanciée directement**.

Elle sert de **modèle** pour d'autres classes.

Elle peut contenir :

- des **méthodes abstraites** (sans corps → à redéfinir dans les classes filles),
- et aussi des **méthodes concrètes** (avec corps → code déjà défini).

👉 Elle représente une **relation "est un"** (exemple : "Un Chien est un Animal").

✅ Exemple :

```
// Classe abstraite
public abstract class Animal
{
    public abstract void FaireDuBruit();  // Méthode abstraite (sans implémen
tation)

    public void Manger()  // Méthode concrète (avec implémentation)
```

```
    {
        Console.WriteLine("L'animal mange.");
    }
}

// Classe fille qui hérite de la classe abstraite
public class Chien : Animal
{
    public override void FaireDuBruit()
    {
        Console.WriteLine("Le chien aboie !");
    }
}
```

🟢 Ici :

- `Animal` est abstraite → on ne peut pas faire `new Animal()`.
- `Chien` hérite de `Animal` et **implémente la méthode abstraite** `FaireDuBruit()`.

⚙️ 2️⃣ Interface

🔷 Définition :

Une **interface** est un **contrat** :

Elle définit **uniquement les signatures** des méthodes (pas de code).

Toute classe qui implémente cette interface **doit définir toutes ses méthodes**.

👉 Elle représente une **relation "peut faire"** (exemple : "Un Chien peut courir").

✅ Exemple :

```
// Interface
public interface ICourir
{
    void Courir();  // Méthode sans implémentation
}

// Classe qui implémente l'interface
```

```
public class Chien : ICourir
{
    public void Courir()
    {
        Console.WriteLine("Le chien court vite !");
    }
}
```

🟢 Ici :

- `ICourir` ne contient que des **méthodes sans corps**.
- `Chien` **doit** implémenter la méthode `Courir()`.

🧠 Q2.

Définis :

- le **polymorphisme par héritage**
- le **polymorphisme par signature**

  ➡️ Donne un exemple de code pour chaque cas.

⚙️ 1️⃣ Polymorphisme par héritage

(→ aussi appelé **polymorphisme par redéfinition** ou *override*)

🔷 Définition :

C'est le fait qu'une **classe fille** redéfinisse une **méthode héritée** de la classe mère pour **adapter son comportement**.

👉 Même nom, même paramètres, mais **comportement différent** selon la classe.

✅ Exemple :

```
using System;

public class Animal
{
```

```
    public virtual void FaireDuBruit()
    {
        Console.WriteLine("L'animal fait un bruit.");
    }
}

public class Chien : Animal
{
    public override void FaireDuBruit()
    {
        Console.WriteLine("Le chien aboie !");
    }
}

public class Chat : Animal
{
    public override void FaireDuBruit()
    {
        Console.WriteLine("Le chat miaule !");
    }
}

class Program
{
    static void Main()
    {
        Animal a1 = new Chien();
        Animal a2 = new Chat();

        a1.FaireDuBruit(); // 🔊 Le chien aboie !
        a2.FaireDuBruit(); // 🔊 Le chat miaule !
    }
}
```

🟢 Ici :

- `FaireDuBruit()` est **redéfinie** dans chaque sous-classe.

- Le comportement dépend **du type réel de l'objet**, pas du type déclaré (`Animal`).

👉 **C'est le polymorphisme dynamique (à l'exécution).**

⚙️ 2️⃣ Polymorphisme par signature

(→ aussi appelé **surcharge de méthode** ou *overloading*)

🔷 Définition :

C'est le fait de **définir plusieurs méthodes portant le même nom** mais avec **des paramètres différents** (nombre, type ou ordre).

👉 Le compilateur choisit **la bonne méthode** selon les **arguments fournis**.

✅ Exemple :

```
using System;

public class Calculatrice
{
    public int Addition(int a, int b)
    {
        return a + b;
    }

    public double Addition(double a, double b)
    {
        return a + b;
    }

    public int Addition(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
```

```
    {
        Calculatrice calc = new Calculatrice();

        Console.WriteLine(calc.Addition(3, 5));      // 8
        Console.WriteLine(calc.Addition(2.5, 4.3));  // 6.8
        Console.WriteLine(calc.Addition(1, 2, 3));   // 6
    }
}
```

🟢 Ici :

- `Addition()` a **le même nom**, mais des **signatures différentes**.
- Le compilateur choisit la version correcte selon les **types et le nombre d'arguments**.

👉 **C'est le polymorphisme statique (à la compilation).**

🧠 Q3.

Qu'est-ce qu'un **délégué** en C# ?

➡️ Donne un exemple d'utilisation avec une **lambda**.

⚙️ 1️⃣ Définition

Un **délégué** est un **type spécial d'objet** qui peut **référencer une méthode**.

👉 En d'autres termes, **c'est comme un "pointeur de fonction"** sécurisé en C#.

Il permet de :

- stocker une méthode dans une variable,
- **passer une méthode en paramètre** à une autre méthode,
- **appeler dynamiquement** une méthode sans connaître son nom exact à l'avance.

⚙️ 2️⃣ Syntaxe générale

```
// Déclaration d'un délégué
public delegate int Operation(int x, int y);
```

- Ici, `Operation` est un **type de délégué** représentant toute **méthode** qui :
  - prend deux `int` en paramètres,
  - et retourne un `int`.

✅ 3️⃣ Exemple classique

```
using System;

public delegate int Operation(int a, int b);

class Program
{
    static void Main()
    {
        // Associer une méthode au délégué
        Operation addition = Additionner;

        // Appeler la méthode via le délégué
        Console.WriteLine(addition(3, 4)); // 🔢 Résultat : 7
    }

    static int Additionner(int x, int y)
    {
        return x + y;
    }
}
```

🟢 Ici :

- `addition` est un **objet délégué** pointant vers la méthode `Additionner`.
- L'appel `addition(3,4)` exécute en réalité `Additionner(3,4)`.

⚙️ 4️⃣ Exemple avec une **lambda**

Les lambdas simplifient beaucoup l'usage des délégués 🪄

```
using System;

public delegate int Operation(int a, int b);

class Program
{
    static void Main()
    {
        // Utilisation d'une expression lambda
        Operation multiplication = (x, y) ⇒ x * y;

        Console.WriteLine(multiplication(5, 6)); // 🔢 Résultat : 30
    }
}
```

🟢 Explication :

- `(x, y) ⇒ x * y` est une **lambda** (fonction anonyme).
- Elle correspond exactement à la signature du délégué `Operation`.
- Pas besoin d'écrire une méthode séparée — tout est en ligne.

🧠 Q4.

Explique la différence entre :

```
Add-Migration
```

et

```
Update-Database
```

dans Entity Framework Core.

⚙️ 1️⃣ Add-Migration

🔷 Définition :

Cette commande sert à **créer un fichier de migration** (un script C#) qui décrit les **changements effectués dans ton modèle de données** (tes classes `Entity`).

> 👉 Elle ne modifie pas encore la base de données, elle prépare seulement les instructions nécessaires pour le faire.

🧩 Exemple :

```
Add-Migration InitialCreate
```

🔍 Ce qui se passe :

- EF Core **compare** ton modèle actuel (classes du code) avec le dernier état connu dans la base.
- Il génère un **fichier de migration** dans le dossier :

  ```
  Migrations/20251028123000_InitialCreate.cs
  ```

- Ce fichier contient des méthodes comme :

  ```
  protected override void Up(MigrationBuilder migrationBuilder)
  {
      migrationBuilder.CreateTable(
          name: "Clients",
          columns: table => new
          {
              Id = table.Column<int>(nullable: false)
                  .Annotation("SqlServer:Identity", "1, 1"),
              Nom = table.Column<string>(nullable: true)
          },
          constraints: table =>
  ```

```
        {
            table.PrimaryKey("PK_Clients", x => x.Id);
        });
    }
```

🟢 Donc :

- `Add-Migration` = “**Prépare les changements**”.
- Rien n’est encore appliqué à la base de données.

⚙️ 2️⃣ Update-Database

🔷 Définition :

Cette commande **exécute les migrations** qui n’ont pas encore été appliquées sur la **base de données réelle**.

> 👉 Elle met à jour le schéma de la base (tables, colonnes, clés, etc.) selon les fichiers de migration.

🧩 Exemple :

```
Update-Database
```

🔍 Ce qui se passe :

- EF Core lit les fichiers dans le dossier `Migrations`.
- Il exécute la méthode `Up()` de chaque migration non encore appliquée.
- Il **modifie la base de données** (création de tables, ajout de colonnes, etc.).
- Il met à jour la table spéciale `__EFMigrationsHistory` pour garder une trace.

🟢 Donc :

- `Update-Database` = “**Applique les changements**”.

🧠 Q5.

Qu’est-ce qu’un **DbContext** ?

➡️ Quelle est la différence entre un `DbSet<Plane>` et une classe `Plane` ?

🧠 Q6.

Explique la différence entre :

- une **méthode d'extension**
- une **méthode normale**

➡️ Donne un exemple concret sur une classe `Passenger`.

⚙️ 1️⃣ Méthode normale

🔷 Définition :

Une **méthode normale** est **définie à l'intérieur même de la classe**.

👉 Elle fait **partie intégrante** de la classe et peut accéder à ses **membres privés**.

✅ Exemple :

```
public class Passenger
{
    public string Name { get; set; }
    public int Age { get; set; }

    // 🔹 Méthode normale
    public void DisplayInfo()
    {
        Console.WriteLine($"Passenger: {Name}, Age: {Age}");
    }
}
```

🧠 Utilisation :

```
Passenger p = new Passenger { Name = "Sami", Age = 25 };
p.DisplayInfo();  // 🔊 Affiche : Passenger: Sami, Age: 25
```

🟢 Ici :

- La méthode `DisplayInfo()` est **membre direct** de `Passenger`.
- Elle a **accès complet** aux attributs `Name` et `Age`.

⚙️ 2️⃣ Méthode d'extension (Extension Method)

🔷 Définition :

Une **méthode d'extension** permet d'**ajouter une méthode à une classe existante, sans la modifier**.

👉 Elle est **définie à l'extérieur** de la classe, mais **appelée comme si elle en faisait partie**.

> C'est pratique quand tu veux enrichir une classe provenant d'une bibliothèque externe (que tu ne peux pas éditer).

✅ Exemple sur la classe `Passenger`

Étape 1️⃣ — Définir la classe d'origine :

```
public class Passenger
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Étape 2️⃣ — Créer une méthode d'extension :

```
public static class PassengerExtensions
{
    // 🔹 Méthode d'extension
    public static bool IsAdult(this Passenger p)
    {
        return p.Age >= 18;
```

```
    }
}
```

> 🧩 Détail important :
>
> - Elle doit être dans une **classe statique**.
> - La méthode doit aussi être **statique**.
> - Le **premier paramètre** commence toujours par le mot-clé `this`, suivi du **type étendu** (`Passenger` ici).

Étape 3️⃣ — Utilisation :

```
Passenger p = new Passenger { Name = "Lina", Age = 17 };

Console.WriteLine($"{p.Name} est adulte ? {p.IsAdult()}");
```

🟢 Résultat :

```
Lina est adulte ? False
```

🟢 Ici :

- Même si `IsAdult()` **n'est pas dans la classe Passenger**, on peut l'appeler comme si elle l'était.
- Le compilateur "injecte" cette méthode au moment de la compilation.

🧠 Q7.

Qu'est-ce qu'une **migration** et à quoi sert-elle ?

➡️ Décris les deux fichiers générés par EF Core après une migration.

Une **migration** en EF Core sert à **synchroniser ton modèle de classes avec la base de données**.

- **Fichier principal** : contient `Up()` (applique les changements) et `Down()` (annule les changements).
- **Fichier snapshot** : représente l'état actuel du modèle pour comparer les modifications futures.

🧠 Q8.

Dans EF Core, à quoi sert l'annotation suivante :

```
[ForeignKey("PlaneId")]
```

➡️ Explique son rôle et son effet dans la base de données.

L'annotation `[ForeignKey("PlaneId")]` en **Entity Framework Core** sert à **indiquer qu'une propriété est une clé étrangère** (foreign key) qui référence une autre entité.

- Elle relie une **relation entre deux tables** dans la base de données.
- Elle permet à EF Core de savoir **quelle colonne utiliser pour la relation**.

💻 PARTIE 2 — CODE FAUX À CORRIGER (avec explication)

Tu trouveras ci-dessous plusieurs codes erronés.

➡️ **Ta mission :** repérer les erreurs, corriger, et expliquer la raison.

❌ Exercice 1 — Classe mal écrite

```
public class Plane
{
    private int Id;
    string Model;
    public Capacity;
```

```
}
```

✅ Corrige :

```
public class Plane
{
    public int Id { get; set; }
    public string Model { get; set; }
    public int Capacity { get; set; }
}
```

💬 Explication :

- Propriétés doivent avoir **type + get/set**
- Le champ `Capacity` n'avait pas de type
- Les attributs doivent être **publics** pour que EF Core les mappe à la base de données.

❌ Exercice 2 — Polymorphisme mal utilisé

```
public class Animal
{
    public void Parler()
    {
        Console.WriteLine("L'animal parle");
    }
}

public class Chien : Animal
{
    public void Parler()
    {
        Console.WriteLine("Le chien aboie");
    }
```

```
}
```

✅ Corrige :

```
public class Animal
{
    public virtual void Parler()
    {
        Console.WriteLine("L'animal parle");
    }
}

public class Chien : Animal
{
    public override void Parler()
    {
        Console.WriteLine("Le chien aboie");
    }
}
```

💬 Explication :

- Sans `virtual` / `override`, il n'y a **pas de polymorphisme**, mais **masquage de méthode**.
- `override` permet de **redéfinir le comportement** de la méthode héritée.

❌ Exercice 3 — Expression lambda incorrecte

```
Func<int> carre = (x) ⇒ x * x;
Console.WriteLine(carre(4));
```

✅ Corrige :

```
Func<int, int> carre = x => x * x;
Console.WriteLine(carre(4));
```

💬 Explication :

- Le délégué `Func<int>` représente une fonction **sans paramètre**.
- Ici, on veut une fonction **avec un paramètre int** → `Func<int, int>`.

❌ Exercice 4 — Requête LINQ invalide

```
var req = Flights.Select f.Destination == "Paris";
```

✅ Corrige :

```
var req = from f in Flights
        where f.Destination == "Paris"
        select f;
```

ou

```
var req = Flights.Where(f => f.Destination == "Paris");
```

💬 Explication :

- En LINQ, il faut utiliser **la syntaxe correcte** (`from...where...select`)

  ou **les méthodes d'extension** (`Where()`, `Select()`).

❌ Exercice 5 — DbContext mal configuré

```
public class AMContext : DbContext
{
```

```
    public DbSet<Plane> Planes { get; set; }

    protected override void OnConfiguring()
    {
        UseSqlServer("Data Source=localhost;Initial Catalog=AMDB;");
    }
}
```

✅ Corrige :

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBu
ilder)
{
    optionsBuilder.UseSqlServer(
        @"Data Source=(localdb)\mssqllocaldb;
          Initial Catalog=AMDB;
          Integrated Security=true");
}
```

💬 Explication :

- La méthode `OnConfiguring` **doit avoir un paramètre** `DbContextOptionsBuilder`
- On doit appeler `optionsBuilder.UseSqlServer(...)`, pas `UseSqlServer(...)` tout seul.

❌ Exercice 6 — Méthode d'extension mal définie

```
public static class StringExtensions
{
    public void Capitalize(this string s)
    {
        s = s.ToUpper();
    }
}
```

✅ Corrige :

```
public static class StringExtensions
{
    public static string Capitalize(this string s)
    {
        return s.ToUpper();
    }
}
```

💬 Explication :

- Une méthode d'extension doit être **static**
- Le premier paramètre doit commencer par `this`
- On ne peut pas modifier directement une chaîne (`string` est immuable) → il faut **retourner** la version modifiée.

❌ Exercice 7 — Migration EF mal faite

```
AddMigration FirstMigration
```

✅ Corrige :

```
Add-Migration FirstMigration
```

💬 Explication :

- Le nom de la commande est **Add-Migration** (avec tiret)
- Si on oublie le tiret, EF Core ne reconnaît pas la commande.

❌ Exercice 8 — Propriété clé étrangère absente

```
public class Flight
{
    public int Id { get; set; }
    public string Destination { get; set; }
    public Plane Plane { get; set; }
}
```

✅ Corrige :

```
public class Flight
{
    public int Id { get; set; }
    public string Destination { get; set; }

    public int PlaneId { get; set; }
    [ForeignKey("PlaneId")]
    public Plane Plane { get; set; }
}
```

💬 Explication :

- Sans la propriété `PlaneId`, EF ne sait pas **quelle colonne utiliser pour la relation**.
- `[ForeignKey]` précise la liaison avec la table `Planes`.

❌ Exercice 9 — Erreur de LINQ avec groupement

```
var req = Flights.GroupBy(f => f.Destination)
                 .Select(g => g.Destination);
```

✅ Corrige :

```
var req = Flights.GroupBy(f ⇒ f.Destination)
                 .Select(g ⇒ g.Key);
```

💬 Explication :

- Après `GroupBy`, chaque `g` est un **groupe** avec une **clé** (`Key`) → on accède via `g.Key`.

❌ Exercice 10 — Erreur de lambda dans EF

```
var longFlights = Flights.Where(f ⇒ f.Duration > "5");
```

✅ Corrige :

```
var longFlights = Flights.Where(f ⇒ f.Duration > 5);
```

💬 Explication :

- `"5"` est une **chaîne de caractères**, pas un **nombre**.
- Il faut comparer des **types identiques** (`double > int` est valide, `double > string` non).